

Algèbre 1 : Chapitre I - Calculs algébriques

Aymen Ben Brik

Chapitre 1

Calculs algébriques

1.1 Sommes et produits finis

1.1.1 Motivation

Dans de nombreux domaines tels que l'analyse de données, l'algorithmique ou les probabilités, nous sommes confrontés à la nécessité d'agréger ou de multiplier de grandes quantités de valeurs. L'écriture explicite de ces opérations (ex : $x_1 + x_2 + \dots + x_{1000}$) devient vite illisible et impraticable. L'introduction des symboles de sommation ($\sum$) et de produit ($\prod$) permet non seulement de compacter l'écriture, mais surtout de manipuler ces blocs de données de manière formelle et rigoureuse pour en extraire de nouvelles propriétés.

1.1.2 Approche par problème : L'astuce du jeune Gauss

Problème : Comment calculer rapidement la somme des 100 premiers nombres entiers : $S = 1 + 2 + 3 + \dots + 99 + 100$?

Résolution : Une approche naïve consisterait à additionner terme à terme. Cependant, si l'on écrit cette somme à l'endroit, puis à l'envers, et qu'on les additionne verticalement :

$$\begin{array}{r} S = 1 + 2 + \dots + 99 + 100 \\ S = 100 + 99 + \dots + 2 + 1 \\ \hline 2S = 101 + 101 + \dots + 101 + 101 \end{array}$$

Puisqu'il y a 100 termes, on obtient $2S = 100 \times 101$, d'où $S = 5050$. Ce problème met en évidence le besoin d'un formalisme (les changements d'indices) pour généraliser ce type de raisonnement à tout entier n .

1.1.3 Définitions

Soient $n, p \in \mathbb{N}$ tels que $p \leq n$, et $(a_k)_{p \leq k \leq n}$ une suite de nombres réels ou complexes.

Définition 1.1 (Somme) : On note par la lettre grecque majuscule Sigma ($\sum$) la somme des éléments a_k pour l'indice k allant de p à n :

$$\sum_{k=p}^n a_k = a_p + a_{p+1} + \dots + a_n$$

Remarque : La variable k est une variable "muette". Elle n'existe qu'à l'intérieur de la somme. $\sum_{k=1}^n a_k = \sum_{i=1}^n a_i$.

Définition 1.2 (Produit) : On note par la lettre grecque majuscule Pi ($\prod$) le produit de ces mêmes éléments :

$$\prod_{k=p}^n a_k = a_p \times a_{p+1} \times \cdots \times a_n$$

Convention : Une somme vide (si $p > n$) vaut 0. Un produit vide vaut 1.

1.1.4 Exemples résolus (Difficulté ascendante)

Exemple 1 (Développement simple - Facile) : Développer $\sum_{k=2}^5 k^2$. *Correction :* On remplace successivement k par 2, 3, 4 et 5.

$$\sum_{k=2}^5 k^2 = 2^2 + 3^2 + 4^2 + 5^2 = 4 + 9 + 16 + 25 = 54$$

Exemple 2 (Changement d'indice - Intermédiaire) : Exprimer $S = \sum_{k=1}^n a_{k+2}$ sous la forme d'une somme dont le terme général est a_j . *Correction :* Posons $j = k + 2$. Quand $k = 1$, $j = 3$. Quand $k = n$, $j = n + 2$. Donc, $S = \sum_{j=3}^{n+2} a_j$.

Exemple 3 (Somme télescopique - Difficile) : Calculer $S_n = \sum_{k=1}^n \left(\frac{1}{k} - \frac{1}{k+1}\right)$. *Correction :* En développant les premiers et derniers termes, on observe des annulations en cascade :

$$S_n = \left(1 - \frac{1}{2}\right) + \left(\frac{1}{2} - \frac{1}{3}\right) + \cdots + \left(\frac{1}{n} - \frac{1}{n+1}\right)$$

Tous les termes intermédiaires s'annulent. Il ne reste que le premier et le dernier :

$$S_n = 1 - \frac{1}{n+1} = \frac{n}{n+1}$$

1.1.5 Exercices pratiques avec Python

Les sommes mathématiques se traduisent naturellement par des boucles `for` en développement logiciel. Python propose également des fonctions intégrées très performantes pour l'analyse de tableaux de données.

```

1  # Exercice : Calculer la somme des carrés des 10 premiers entiers
2
3  # Methode 1 : Approche algorithmique classique (boucle for)
4  somme_boucle = 0
5  for k in range(1, 11): # k va de 1 a 10
6      somme_boucle += k**2
7  print(f"Resultat avec boucle : {somme_boucle}")
8
9  # Methode 2 : Approche "Pythonique" (comprehension de liste)
10 # Tres utile pour manipuler rapidement des donnees
11 somme_rapide = sum([k**2 for k in range(1, 11)])
12 print(f"Resultat avec sum() : {somme_rapide}")

```

1.1.6 Exercices à faire

1. **Exercice 1** : Calculer la valeur exacte de $\prod_{k=1}^4 (2k)$.
2. **Exercice 2** : En utilisant la relation de Chasles, simplifier l'expression : $\sum_{k=1}^{100} k^3 - \sum_{k=1}^{98} k^3$.
3. **Exercice 3** : Montrer par le calcul que $\sum_{k=0}^n (2k+1) = (n+1)^2$.
4. **Exercice 4 (Code)** : Écrire une fonction Python `factorielle(n)` qui calcule $n! = \prod_{k=1}^n k$ à l'aide d'une boucle.

1.2 Sommes doubles

1.2.1 Motivation

Jusqu'à présent, nous avons sommé les éléments d'une liste à une dimension. Cependant, en algèbre linéaire (avec les matrices) ou en analyse de données (avec les tableaux et les images numériques), les données sont souvent organisées en deux dimensions : par lignes et par colonnes. Pour agréger ou manipuler ces données, nous devons itérer sur deux indices simultanément. La maîtrise des sommes doubles permet de naviguer mathématiquement dans ces grilles avec une précision absolue.

1.2.2 Approche par problème : Lignes ou colonnes en premier ?

Problème : Imaginons un tableau de nombres rectangulaire comportant n lignes et p colonnes. On note $a_{i,j}$ le nombre situé à la ligne i et à la colonne j . Comment calculer la somme totale de tous les nombres de ce tableau ?

Résolution : Deux stratégies intuitives s'offrent à nous :

1. **Par lignes** : On calcule d'abord la somme de chaque ligne i (soit $L_i = \sum_{j=1}^p a_{i,j}$), puis on additionne le total de toutes les lignes : $\sum_{i=1}^n L_i$.
2. **Par colonnes** : On calcule d'abord la somme de chaque colonne j (soit $C_j = \sum_{i=1}^n a_{i,j}$), puis on additionne ces totaux : $\sum_{j=1}^p C_j$.

Ces deux méthodes comptent exactement les mêmes éléments. On en déduit une propriété fondamentale d'interversion :

$$\sum_{i=1}^n \left(\sum_{j=1}^p a_{i,j} \right) = \sum_{j=1}^p \left(\sum_{i=1}^n a_{i,j} \right)$$

1.2.3 Définitions

Définition 2.1 (Somme sur un rectangle) : Si les indices i et j varient indépendamment l'un de l'autre ($1 \leq i \leq n$ et $1 \leq j \leq p$), on note la somme double :

$$\sum_{i=1}^n \sum_{j=1}^p a_{i,j} \quad \text{ou plus simplement} \quad \sum_{\substack{1 \leq i \leq n \\ 1 \leq j \leq p}} a_{i,j}$$

Définition 2.2 (Somme sur un triangle / Indices dépendants) : Si la borne d'un indice dépend de l'autre (par exemple $1 \leq i \leq j \leq n$), on somme uniquement sur

une "moitié" du tableau (au-dessus ou en dessous de la diagonale). L'ordre de sommation devient crucial :

$$\sum_{1 \leq i \leq j \leq n} a_{i,j} = \sum_{j=1}^n \left(\sum_{i=1}^j a_{i,j} \right) = \sum_{i=1}^n \left(\sum_{j=i}^n a_{i,j} \right)$$

Règle d'or : La somme extérieure ne doit jamais avoir d'indice variable dans ses bornes.

1.2.4 Exemples résolus (Difficulté ascendante)

Exemple 1 (Variables séparables - Facile) : Calculer $S = \sum_{i=1}^n \sum_{j=1}^p (i \times j)$.
Correction : Puisque les termes dépendants de i sont des constantes par rapport à j , on peut factoriser :

$$S = \sum_{i=1}^n \left(i \sum_{j=1}^p j \right) = \left(\sum_{i=1}^n i \right) \times \left(\sum_{j=1}^p j \right) = \frac{n(n+1)}{2} \times \frac{p(p+1)}{2}$$

Exemple 2 (Somme triangulaire simple - Intermédiaire) : Calculer $S = \sum_{i=1}^n \sum_{j=i}^n 1$. (Cela revient à compter le nombre d'éléments dans un triangle). *Correction* : On commence par la somme intérieure (sur j) :

$$\sum_{j=i}^n 1 = (n - i + 1) \quad (\text{nombre de termes})$$

On remplace dans la somme extérieure :

$$S = \sum_{i=1}^n (n - i + 1) = \sum_{i=1}^n n - \sum_{i=1}^n i + \sum_{i=1}^n 1 = n^2 - \frac{n(n+1)}{2} + n = \frac{n(n+1)}{2}$$

Exemple 3 (Inversion de l'ordre de sommation - Difficile) : Soit $S = \sum_{i=1}^n \sum_{j=i}^n \frac{i}{j}$. Il est difficile de sommer d'abord sur j . Invertissons les sommes. *Correction* : Le domaine est $1 \leq i \leq j \leq n$. Si on somme d'abord sur i , pour un j fixé, i varie de 1 à j . L'indice j varie de 1 à n .

$$S = \sum_{j=1}^n \sum_{i=1}^j \frac{i}{j} = \sum_{j=1}^n \frac{1}{j} \left(\sum_{i=1}^j i \right) = \sum_{j=1}^n \frac{1}{j} \frac{j(j+1)}{2} = \frac{1}{2} \sum_{j=1}^n (j+1)$$

$$S = \frac{1}{2} \left(\frac{n(n+1)}{2} + n \right) - \frac{1}{2} \quad (\text{en ajustant l'indice pour } j=1)$$

1.2.5 Exercices pratiques avec Python

En programmation, une somme double se traduit par des boucles imbriquées. En analyse de données, la bibliothèque **numpy** permet d'optimiser ces calculs matriciels.

```
1 import numpy as np
2
3 # Creation d'un tableau (matrice) 3x3
4 matrice = np.array([
5     [1, 2, 3],
6     [4, 5, 6],
```

```

7         [7, 8, 9]
8     ])
9
10    # Methode 1 : Somme double avec boucles imbriquees (Algorithmique
      classique)
11    somme_totale = 0
12    for i in range(3):          # Lignes
13        for j in range(3):      # Colonnes
14            somme_totale += matrice[i][j]
15    print(f"Somme_via_boucles : {somme_totale}")
16
17    # Methode 2 : Somme triangulaire (i <= j, au-dessus de la diagonale
      )
18    somme_triangle = 0
19    for i in range(3):
20        for j in range(i, 3):    # j démarre a i
21            somme_triangle += matrice[i][j]
22    print(f"Somme_triangulaire : {somme_triangle}")
23
24    # Methode 3 : L'approche matricielle (Python pour la data)
25    print(f"Somme_rapide_avec_numpy : {np.sum(matrice)}")

```

1.2.6 Exercices à faire

1. **Exercice 1** : Calculer la valeur de $\sum_{i=1}^n \sum_{j=1}^n (i + j)$.
2. **Exercice 2** : Inverser l'ordre des sommes dans l'expression suivante et la calculer : $\sum_{i=1}^n \sum_{j=i}^n 2^j$.
3. **Exercice 3** : Que vaut la somme double $\sum_{1 \leq i, j \leq n} \min(i, j)$? (Indice : séparez les cas $i \leq j$ et $j < i$).

1.3 Formule du Binôme

1.3.1 Motivation : Analyse de séquences binaires

En probabilités et en analyse de données, il est fréquent d'étudier des séquences d'événements binaires. Prenons l'exemple de l'analyse des performances dans un match de football : une passe tentée par un joueur est soit réussie (notons-la R), soit manquée (notons-la M). Si l'on souhaite analyser les statistiques d'un joueur qui tente n passes durant un match, il existe une multitude de scénarios possibles menant à exactement k passes réussies. La formule du binôme est l'outil algébrique qui permet de dénombrer instantanément ces combinaisons et de modéliser cette distribution sans avoir à lister manuellement chaque trajectoire possible.

1.3.2 Approche par problème : L'arbre des passes

Problème : Un milieu de terrain tente 3 passes consécutives. En représentant algébriquement chaque tentative par la somme $(R + M)$, comment déduire tous les scénarios possibles ?

Résolution : Si l'on développe les tentatives successives, on obtient :

- 1 passe : $(R + M)^1 = R + M$ (1 scénario 100% réussi, 1 scénario 100% raté)
- 2 passes : $(R + M)^2 = R^2 + 2RM + M^2$ (1 avec 2 réussites, 2 avec 1 réussite/1 échec, 1 avec 2 échecs)
- 3 passes : $(R + M)^3 = R^3 + 3R^2M + 3RM^2 + M^3$

Si le joueur tente 10 passes, développer $(R + M)^{10}$ par multiplications successives devient laborieux. Or, en observant les coefficients, on remarque une structure géométrique :

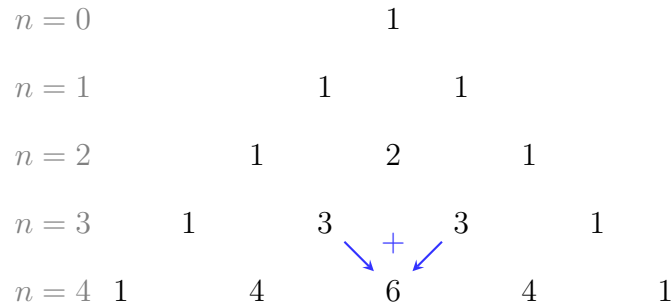


FIGURE 1.1 – Le Triangle de Pascal. Chaque nombre est la somme des deux situés juste au-dessus.

Chaque nombre y est la somme des deux nombres situés juste au-dessus de lui. Pour généraliser cela à n passes, il nous faut définir rigoureusement ces coefficients.

1.3.3 Définitions

Définition 3.1 (Factorielle) : Soit $n \in \mathbb{N}$. On appelle factorielle n , notée $n!$, le produit des n premiers entiers naturels non nuls :

$$n! = 1 \times 2 \times \cdots \times n = \prod_{k=1}^n k$$

Convention : $0! = 1$.

Définition 3.2 (Coefficient binomial) : Pour $n \in \mathbb{N}$ et $k \in \mathbb{N}$ tels que $0 \leq k \leq n$, on définit le coefficient binomial "k parmi n", noté $\binom{n}{k}$, par :

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Il représente le nombre de façons de choisir k succès parmi n tentatives (par exemple, choisir quelles 3 passes ont été réussies parmi 10).

Théorème (Formule du Binôme de Newton) : Pour tous nombres complexes (ou réels) a et b , et pour tout entier naturel n :

$$(a + b)^n = \sum_{k=0}^n \binom{n}{k} a^k b^{n-k}$$

1.3.4 Exemples résolus (Difficulté ascendante)

Exemple 1 (Développement direct - Facile) : Développer $(x + 1)^4$. *Correction :* En appliquant la formule avec $a = x$, $b = 1$ et $n = 4$:

$$(x + 1)^4 = \sum_{k=0}^4 \binom{4}{k} x^k 1^{4-k} = \binom{4}{0} x^0 + \binom{4}{1} x^1 + \binom{4}{2} x^2 + \binom{4}{3} x^3 + \binom{4}{4} x^4$$

En calculant les coefficients (ligne $n = 4$ du triangle : 1, 4, 6, 4, 1), on obtient :

$$(x + 1)^4 = 1 + 4x + 6x^2 + 4x^3 + x^4$$

Exemple 2 (Recherche de coefficient - Intermédiaire) : Quel est le coefficient de x^3 dans le développement de $(2x - 3)^5$? *Correction :* Le terme général du développement est $\binom{5}{k} (2x)^k (-3)^{5-k}$. Pour isoler x^3 , on choisit $k = 3$. Le terme correspondant est $\binom{5}{3} (2x)^3 (-3)^2 = 10 \times 8x^3 \times 9 = 720x^3$. Le coefficient recherché est 720.

Exemple 3 (Calcul de somme via le binôme - Difficile) : Calculer la somme $S = \sum_{k=0}^n \binom{n}{k}$. *Correction :* On pose astucieusement $a = 1$ et $b = 1$ dans la formule du binôme :

$$(1 + 1)^n = \sum_{k=0}^n \binom{n}{k} 1^k 1^{n-k} = \sum_{k=0}^n \binom{n}{k}$$

On en déduit immédiatement que $S = 2^n$.

1.3.5 Exercices pratiques avec Python

Les factorielles grandissent de manière exponentielle. En Python, plutôt que de coder la division des factorielles à la main, il est recommandé d'utiliser les fonctions optimisées de la bibliothèque standard pour éviter les lenteurs et les dépassements de capacité sur de grandes données.

```
1 import math
2
3 # Calcul du nombre de combinaisons (ex: 3 passes reussies parmi 10)
4 n_tentatives = 10
5 k_succes = 3
6 combinaisons = math.comb(n_tentatives, k_succes)
7 print(f"Scenarios possibles ({k_succes} succes sur {n_tentatives}) : {combinaisons}")
8
9 # Fonction pour generer et afficher le Triangle de Pascal
10 def afficher_pascal(lignes):
11     for n in range(lignes):
12         ligne_actuelle = [math.comb(n, k) for k in range(n + 1)]
13         # Affichage centre pour visualiser la symetrie
14         print(" ".join(map(str, ligne_actuelle)).center(40))
15
16 print("\nTriangle de Pascal (n=0 a n=5) :")
17 afficher_pascal(6)
```


1.3.6 Exercices à faire

1. **Exercice 1** : Calculer la somme alternée : $\sum_{k=0}^n (-1)^k \binom{n}{k}$.
2. **Exercice 2** : Développer l'expression $(x - \frac{1}{x})^6$ et simplifier les termes.
3. **Exercice 3** : Démontrer la formule de Pascal : $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ (pour $1 \leq k \leq n-1$) en utilisant la définition algébrique avec les factorielles.

Pour aller plus loin : L'Algèbre au cœur de la Data Science

Bien que les concepts de sommes doubles et de coefficients binomiaux puissent sembler purement théoriques de prime abord, ils constituent en réalité le moteur algorithmique des technologies modernes d'analyse de données. Pour clôturer ce premier chapitre, je vous invite à explorer les deux pistes de recherche suivantes.

Piste 1 : Sommes doubles et Vision par ordinateur (Computer Vision)

Lorsqu'un algorithme de vision par ordinateur analyse un flux vidéo (par exemple, pour détecter automatiquement la balle ou traquer les déplacements des joueurs sur un terrain de football), il traite chaque image comme une immense matrice de pixels.

Pour détecter des contours, des lignes ou des mouvements, les algorithmes utilisent une opération mathématique appelée **convolution**. Appliquer un filtre de convolution sur un pixel de coordonnées (x, y) revient exactement à calculer une double somme sur un voisinage de ce pixel, pondérée par une petite matrice appelée "noyau" (F) :

$$Nouvelle_Valeur_{x,y} = \sum_{i=-1}^1 \sum_{j=-1}^1 F_{i,j} \times Pixel_{x+i,y+j}$$

Sujet de recherche : Renseignez-vous sur les "Noyaux de convolution" (*Convolution Kernels*) tels que les filtres de Sobel. Vous découvrirez comment de simples sommes doubles en algèbre permettent aux ordinateurs de "voir" les formes.

Piste 2 : Combinaisons, Scraping et "Fléau de la dimension"

Imaginez que vous ayez constitué une vaste base de données (par exemple, en automatisant l'extraction web de milliers d'offres d'emploi). Chaque offre est décrite par n caractéristiques ou variables (salaire, localisation, mots-clés des compétences, etc.).

Si vous souhaitez entraîner un modèle d'Intelligence Artificielle pour prédire si une offre trouvera rapidement preneur, vous devrez choisir le meilleur sous-ensemble de variables. Or, si vous avez $n = 50$ variables et que vous souhaitez tester toutes les combinaisons possibles de $k = 5$ variables, le nombre de modèles à entraîner est exactement le coefficient binomial :

$$\binom{50}{5} = 2\,118\,760 \text{ combinaisons}$$

C'est ce que l'on appelle en Data Science le **fléau de la dimension** (*Curse of dimensionality*) ou l'explosion combinatoire.

Sujet de recherche : Explorez les algorithmes de **sélection de caractéristiques** (*Feature Selection*) en Machine Learning. Comment les data scientists font-ils pour trouver le meilleur modèle sans avoir à calculer ces millions de combinaisons dictées par la formule du binôme ?

Travaux Dirigés : Calculs Algébriques

Partie A : Entraînement technique

Exercice 1 : Manipulations de sommes simples

1. Calculer la somme télescopique suivante en fonction de n :

$$S_n = \sum_{k=1}^n \ln \left(\frac{k+1}{k} \right)$$

2. En utilisant un changement d'indice, démontrer que :

$$\sum_{k=1}^n (a_k - a_{k-1}) = a_n - a_0$$

Exercice 2 : Sommes doubles et inversions

1. Calculer la somme double sur un rectangle :

$$A_n = \sum_{i=1}^n \sum_{j=1}^n (2i + j)$$

2. Calculer la somme sur un domaine triangulaire en inversant l'ordre de sommation :

$$B_n = \sum_{i=1}^n \sum_{j=i}^n j$$

Exercice 3 : Autour du Binôme et de Pascal

1. En utilisant la formule du binôme de Newton, simplifier la somme : $\sum_{k=0}^n \binom{n}{k} 3^k$.
2. En dérivant la fonction $f(x) = (1+x)^n$, démontrer la formule suivante (très utile en probabilités) :

$$\sum_{k=0}^n k \binom{n}{k} = n 2^{n-1}$$

Partie B : Modélisation et Algorithmique

Exercice 4 : Extraction de données (Web Scraping)

Un script Python automatisé parcourt n pages d'un portail web pour extraire des offres d'emploi. L'algorithme est conçu tel que sur la page numéro k , il parvient à extraire exactement $3k - 1$ offres valides.

1. Écrire la somme E_n représentant le nombre total d'offres d'emploi extraites après le parcours de n pages.
2. Calculer cette somme en fonction de n .
3. Combien d'offres seront stockées dans la base de données si le script parcourt 50 pages ?

Exercice 5 : Vision par ordinateur et analyse sportive

Lors de l'analyse vidéo d'un match de football par vision par ordinateur, on isole une zone spécifique du terrain représentée par une matrice de pixels M de taille $n \times n$. Pour détecter le mouvement des joueurs, l'algorithme calcule un "poids analytique" pour chaque coordonnée (i, j) défini par $P_{i,j} = i \times j$. Afin d'optimiser le temps de calcul, le script ne balaie que le triangle supérieur de l'image, c'est-à-dire les pixels où $i \leq j$.

1. Écrire la somme double W_n représentant le poids analytique total de la zone balayée.
2. Calculer cette somme en commençant par sommer sur l'indice i . *Indice : Rappelez-vous que $\sum_{i=1}^j i = \frac{j(j+1)}{2}$ et $\sum_{j=1}^n j^3 = \left(\frac{n(n+1)}{2}\right)^2$.*

Exercice 6 : Combinaisons de variables

Dans la phase de préparation de nos données d'analyse, nous disposons d'un ensemble de n variables distinctes (vitesse, accélération, position, etc.). Nous voulons tester des modèles prédictifs en essayant toutes les combinaisons possibles de ces variables (des modèles incluant 0 variable jusqu'aux modèles incluant les n variables).

1. Exprimer le nombre de modèles contenant exactement k variables.
2. Montrer, à l'aide de la formule du binôme, que le nombre total de modèles à entraîner sera exactement de 2^n . (Ce résultat illustre la notion d'explosion combinatoire en machine learning).

Travaux Pratiques (TP) Python

L'objectif de ce TP est de transformer les outils théoriques de ce chapitre (sommes, produits, binôme) en algorithmes opérationnels, et de les appliquer à des problèmes de probabilités, d'analyse numérique et de calcul symbolique. Toutes les implémentations seront réalisées en Python (modules `math`, `itertools`, `matplotlib` optionnel).

Exercice 1 : Sommes et produits *from scratch*

1. **[Bloom : Compréhension | Difficulté : Facile]**
Écrire deux fonctions `somme(L)` et `produit(L)` qui prennent une liste de nombres et retournent respectivement leur somme et leur produit, sans utiliser `sum()` ni `math.prod()`.

2. **[Bloom : Application | Difficulté : Facile]**

En déduire deux fonctions `somme_indices(f, p, n)` et `produit_indices(f, p, n)` qui calculent respectivement $\sum_{k=p}^n f(k)$ et $\prod_{k=p}^n f(k)$ pour une fonction f donnée et des bornes $p \leq n$. Tester avec $f(k) = k^2$, $p = 1$, $n = 10$.

Exercice 2 : Sommes télescopiques

1. **[Bloom : Application | Difficulté : Facile]**

Calculer $S_n = \sum_{k=1}^n \frac{1}{k(k+1)}$ numériquement pour $n = 10, 100, 1000$ et 10^6 .

2. **[Bloom : Analyse | Difficulté : Moyenne]**

Démontrer (à la main) que $S_n = 1 - \frac{1}{n+1}$, puis vérifier numériquement la formule en comparant les deux calculs et en mesurant l'erreur relative.

3. **[Bloom : Évaluation | Difficulté : Moyenne]**

Comparer le temps d'exécution des deux méthodes pour $n = 10^7$. Quelle est la complexité de chaque approche ?

Exercice 3 : Sommes doubles et ordre des boucles

1. **[Bloom : Application | Difficulté : Moyenne]**

Implémenter $T(n) = \sum_{i=1}^n \sum_{j=1}^i (i+j)$ avec deux fonctions : `T_externe_i(n)` (boucle externe sur i) et `T_externe_j(n)` (boucle externe sur j , en réorganisant le domaine de sommation).

2. **[Bloom : Évaluation | Difficulté : Moyenne]**

Vérifier numériquement que les deux versions donnent le même résultat pour $n = 100$. Comparer les temps d'exécution pour $n = 5000$.

3. **[Bloom : Synthèse | Difficulté : Difficile]**

Démontrer la formule fermée $T(n) = \frac{n(n+1)(2n+1)}{6} + \frac{n^2(n+1)}{4}$ (en utilisant les formules de $\sum k$ et $\sum k^2$) et vérifier numériquement.

Exercice 4 : Triangle de Pascal

1. **[Bloom : Application | Difficulté : Facile]**

Écrire une fonction `pascal(N)` qui retourne les $N+1$ premières lignes du triangle de Pascal sous forme de liste de listes, en utilisant uniquement la relation $\binom{n+1}{k} = \binom{n}{k-1} + \binom{n}{k}$.

2. **[Bloom : Application | Difficulté : Moyenne]**

En utilisant le triangle, écrire une fonction `coefficient_binomial(n, k)` qui ne calcule pas les factorielles. Comparer son temps d'exécution à `math.comb(n, k)` pour $n = 1000$.

3. **[Bloom : Analyse | Difficulté : Moyenne]**

Vérifier numériquement les deux propriétés sur les lignes du triangle : symétrie $\binom{n}{k} = \binom{n}{n-k}$, et somme $\sum_{k=0}^n \binom{n}{k} = 2^n$.

Exercice 5 : Récursion vs formule fermée pour $\binom{n}{k}$

1. **[Bloom : Application | Difficulté : Moyenne]**

Implémenter `binom_recuratif(n, k)` avec la relation de Pascal et les conditions aux bords ($\binom{n}{0} = \binom{n}{n} = 1$). Implémenter aussi `binom_factoriel(n, k)` via $\frac{n!}{k!(n-k)!}$.

2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Mesurer le temps d'exécution des deux versions pour $\binom{30}{15}$. La version récursive sans mémorisation est-elle utilisable ?
3. **[Bloom : Synthèse | Difficulté : Difficile]**
Optimiser la version récursive avec `functools.lru_cache` et comparer les performances. Donner la complexité asymptotique de chaque version.

Exercice 6 : Identité de Vandermonde

1. **[Bloom : Synthèse | Difficulté : Moyenne]**
L'identité de Vandermonde affirme :

$$\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}.$$

Écrire une fonction `verifier_vandermonde(m, n, r)` qui calcule les deux membres et retourne `True` ssi ils sont égaux.

2. **[Bloom : Création | Difficulté : Difficile]**
Tester l'identité pour 50 triplets aléatoires (m, n, r) avec $m, n \leq 30$ et $0 \leq r \leq m+n$. Conclure.

Exercice 7 : Loi binomiale et application en probabilités

1. **[Bloom : Application | Difficulté : Moyenne]**
Si X suit une loi binomiale $\mathcal{B}(n, p)$, alors $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$. Implémenter `proba_binomiale(n, k, p)`.
2. **[Bloom : Évaluation | Difficulté : Moyenne]**
Vérifier numériquement que $\sum_{k=0}^n P(X = k) = 1$ pour $(n, p) \in \{(10, 0.3), (50, 0.7), (100, 0.5)\}$. Cette vérification est une conséquence directe de quelle formule ?
3. **[Bloom : Création | Difficulté : Difficile]**
Avec `matplotlib`, tracer la distribution $P(X = k)$ en fonction de k pour $(n, p) = (20, 0.3)$. Calculer aussi l'espérance théorique $E[X] = np$ et la variance $V[X] = np(1-p)$.

Exercice 8 : Développement symbolique du binôme

1. **[Bloom : Création | Difficulté : Difficile]**
Écrire une fonction `developeur_binome(n)` qui retourne, sous forme de chaîne de caractères, le développement de $(a+b)^n$ selon la formule du binôme. Exemple attendu : `developeur_binome(3)` retourne `"a^3 + 3*a^2*b + 3*a*b^2 + b^3"`.
2. **[Bloom : Synthèse | Difficulté : Difficile]**
Comparer la sortie avec `sympy.expand((a+b)**n)` pour $n = 5$. Discuter des limites de votre implémentation comparée au moteur symbolique.